



Security Audit Report

BeeZee (BZE)

Phase 2

v1.0

March 18, 2026

Table of Contents

Table of Contents	2
License	3
Disclaimer	4
Introduction	5
Purpose of This Report	5
Codebase Submitted for the Audit	5
Methodology	6
Functionality Overview	6
How to Read This Report	8
Code Quality Criteria	9
Summary of Findings	10
Detailed Findings	12
1. Unbounded epoch hook processing enabling chain halt via raffle expiration flooding	12
2. Unbounded weekly burn hook allowing consensus-level denial of service via denom flooding	13
3. Incomplete genesis validation allowing malformed or duplicate factory denominations	14
4. Missing upper bound validation on the tax parameter	14
5. Event emission uses stale article ID resulting in incorrect observability data	15
6. Incomplete genesis validation allowing structurally invalid or inconsistent cointrunk module state	15
7. Proposer influencable deterministic randomness enabling MEV and raffle outcome manipulation	16
8. Misleading expiration check in raffle join logic	17
9. Incomplete genesis validation allowing malformed raffle and burn state initialization	18
10. Unbounded EndBlock participant processing creating potential liveness risk	18
11. Epochs BeginBlocker processes only one tick per block, leaving epoch-dependent state stale after chain halt	19
12. Optional TradeKeeper dependency used without nil safety checks	20
13. Incorrect telemetry metric key used in EndBlock instrumentation	21
14. No minimum pot requirement allows cheap raffle squatting per denom	21
15. Raffle participant counter tracks tickets instead of unique participants	22
16. No article URL uniqueness check allows anonymous duplicate spam	22

License



THIS WORK IS LICENSED UNDER A [CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE](https://creativecommons.org/licenses/by-nc/4.0/).

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUCT ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

<https://oaksecurity.io/>
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by BeeZee to perform a security audit of BeeZee (BZE).

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/bze-alphateam/bze
Commit	d6c1b6358cb2b20759ff14dba3b359fa028affeb
Scope	The scope is restricted to the following folders: • x/burner • x/cointrunk • x/epochs • x/tokenfactory

Fixes verified at commit	3c50ee928d453da384f64a0cd1e981b2e2ce9795 Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.
--------------------------	--

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
 - a. Race condition analysis
 - b. Under-/overflow issues
 - c. Key management vulnerabilities
4. Report preparation

Functionality Overview

BeeZee is a Cosmos SDK based blockchain that integrates on-chain trading, incentive distribution, and fee management at the protocol level.

The burner module implements the supply-reduction and raffle subsystem: it accumulates coins sent by users and other modules for destruction, periodically burns native and token-factory denoms while permanently locking LP tokens and attempting to route IBC assets through the trading subsystem, records burned amounts per block height, and runs on-chain raffles whose ticket payments fund prize pots and whose remaining balances are burned when the raffle expires.

The cointrunk module provides a curated content and tipping subsystem: it maintains a whitelist of accepted domains and a registry of publishers, stores submitted articles and per-period counters for anonymous posts, enforces fees and limits on anonymous publishing by directing those fees to the community pool, and processes “pay publisher respect” tips that split payments between publishers and the community pool while tracking publisher reputation.

The epochs module provides generic time-based scheduling for the protocol: it defines named epoch streams such as hour, day, and week, tracks their progression from genesis using block time, emits start and end events for each epoch, and invokes registered hooks so other modules (including rewards, burner, and cointrunk) can execute periodic logic such as payouts, burns, cleanups, and counter resets.

The tokenfactory module powers user-defined asset issuance: it lets accounts create custom factory denoms under their address, records per-denom admin authority, exposes admin-gated operations for minting, burning, transferring admin, and setting denom metadata, and charges a configurable creation fee that is captured, swapped via the trading subsystem into the native token, and forwarded into the fee-collection pipeline.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium	The cosmos appchain includes multiple modules with hooks and ABCI methods.
Code readability and clarity	Medium-High	-
Level of documentation	Medium-High	-
Test coverage	Low	Test coverage cannot be computed due to errors during test execution.

Summary of Findings

No	Description	Severity	Status
1	Unbounded epoch hook processing enabling chain halt via raffle expiration flooding	Critical	Resolved
2	Unbounded weekly burn hook allowing consensus-level denial of service via denom flooding	Critical	Partially Resolved
3	Incomplete genesis validation allowing malformed or duplicate factory denominations	Minor	Acknowledged
4	Missing upper bound validation on the tax parameter	Minor	Resolved
5	Event emission uses stale article ID resulting in incorrect observability data	Minor	Resolved
6	Incomplete genesis validation allowing structurally invalid or inconsistent cointrunk module state	Minor	Acknowledged
7	Proposer influencable deterministic randomness enabling MEV and raffle outcome manipulation	Minor	Acknowledged
8	Misleading expiration check in raffle join logic	Minor	Resolved
9	Incomplete genesis validation allowing malformed raffle and burn state initialization	Minor	Acknowledged
10	Unbounded EndBlock participant processing creating potential liveness risk	Minor	Resolved
11	Epochs BeginBlocker processes only one tick per block, leaving epoch-dependent state stale after chain halt	Minor	Resolved
12	Optional TradeKeeper dependency used without nil safety checks	Informational	Resolved
13	Incorrect telemetry metric key used in EndBlock instrumentation	Informational	Resolved
14	No minimum pot requirement allows cheap raffle squatting per denom	Informational	Resolved
15	Raffle participant counter tracks tickets instead of unique participants	Informational	Acknowledged

16	No article URL uniqueness check allows anonymous duplicate spam	Informational	Acknowledged
----	---	---------------	--------------

Detailed Findings

1. Unbounded epoch hook processing enabling chain halt via raffle expiration flooding

Severity: Critical

The `GetBurnerRaffleCleanupHook` function in `x/burner/keeper/hooks.go` registers an `AfterEpochEnd` hook for the "hour" epoch identifier, invoking `burnerRaffleCleanup(ctx, epochNumber)` whenever a raffle epoch concludes. The cleanup routine performs unbounded and attacker-influencable work during each execution.

For a given `epochNumber`, the function retrieves all raffles whose `EndAt` matches that epoch via `GetRaffleDeleteHookByEndAtPrefix`. It then iterates over each raffle and executes multiple state-intensive operations, including deleting the raffle record, loading and iterating over all winners for the associated denom (up to 100 entries per raffle), loading the raffle module account, querying its full balance for the denom, invoking `BurnAnyCoins` (which may itself iterate over multiple denoms and interact with other modules such as a trade module), updating burned coin accounting state, and emitting a `RaffleFinishedEvent`.

There is no upper bound on the number of raffles that may share the same `EndAt` value. A malicious actor can therefore create a large number of raffles, potentially using numerous low-value tokenfactory denoms, and intentionally configure them to expire in the same hourly epoch. When that epoch ends, the hook must process all expired raffles within a single callback execution, resulting in a potentially massive amount of KV store I/O, balance queries, inter-module calls, and event emissions.

Because epoch hooks execute in ABCI outside normal per-transaction gas limits, this unbounded loop can cause epoch processing within `x/epochs'` `BeginBlocker` to exceed consensus timeouts.

This may result in repeated failures to finalize blocks that cross the affected epoch boundary, effectively causing a chain-wide denial of service and potentially halting the network.

Recommendation

We recommend introducing strict bounds and defensive controls on epoch hook processing. This may include limiting the number of raffles processed per epoch callback and introducing pagination or batching mechanisms across multiple blocks.

Status: Resolved

2. Unbounded weekly burn hook allowing consensus-level denial of service via denom flooding

Severity: Critical

The `GetBurnerPeriodicBurnHook` function in `x/burner/keeper/hooks.go` registers an `AfterEpochEnd` hook for the "week" epoch identifier.

When `epochNumber % params.PeriodicBurningWeeks == 0`, the hook invokes `burnModuleCoins`, which retrieves all balances held by the burner module account and passes them to `BurnAnyCoins(ctx, types.ModuleName, allCoins)`.

An adversary can continuously transfer numerous low-value or "dust" coins to the burner module account, including a wide range of IBC denominations, tokenfactory denoms, or other assets. Because all balances are processed together when the weekly epoch fires, the cleanup operation must iterate over the entire balance set in a single epoch callback.

During execution, `BurnAnyCoins` iterates through every coin and classifies each denomination using logic such as `filterCoinsToBurn`, which may invoke `tradeKeeper.IsNativeDenom`, `CanSwapForNativeDenom`, `bzeutils.IsTokenFactoryDenom`, and `IsLpTokenDenom`.

For exchangeable or lockable coins, it may further call into `ModuleAddLiquidityWithNativeDenom` and `SendCoinsFromModuleToModule`, resulting in additional cross-module state transitions and bank operations. The total computational and I/O cost, therefore, scales linearly with the number of distinct denominations and balances stored in the module account.

There is no upper bound on the number of denominations that can be accumulated. By repeatedly sending a large variety of small-balance coins to the burner account, an attacker can force the weekly hook to perform extensive iteration and inter-module processing in a single epoch execution.

Because epoch hooks are executed in ABCI and are not constrained by per-transaction gas limits, this unbounded workload can cause the `BeginBlocker` execution during the targeted weekly epoch to exceed consensus timeouts. In extreme cases, this may result in repeated block finalization failures at that height, triggering consensus timeouts and potentially causing a chain halt.

Recommendation

We recommend introducing explicit bounds and batching mechanisms for periodic burn processing. This may include limiting the number of denominations processed per epoch or distributing burn operations across multiple blocks.

Status: Partially Resolved

3. Incomplete genesis validation allowing malformed or duplicate factory denominations

Severity: Minor

The `GenesisState.Validate` implementation in `x/tokenfactory/types/genesis.go` only invokes `gs.Params.Validate` and omits validation of the `FactoryDenoms` slice.

As a result, the genesis file may contain duplicate factory denominations, improperly formatted token factory denoms (i.e., not conforming to the expected `factory/{creator}/{subdenom}` structure), or entries with malformed or invalid Bech32 admin/authority addresses.

Because these invariants are not enforced during genesis validation, a misconfigured deployment process could initialize the chain with inconsistent or conflicting state.

This may lead to conflicting or duplicated denomination entries, invalid authority assignments enabling unauthorized control over token factory assets, or violation of module invariants that should be enforced at chain initialization time.

Recommendation

We recommend extending `GenesisState.Validate` to perform a comprehensive validation of the `FactoryDenoms` slice during genesis processing.

Status: Acknowledged

4. Missing upper bound validation on the tax parameter

Severity: Minor

The `validatePublisherRespectParams` function in `x/cointrunk/types/params.go` validates that `PublisherRespectParams.Tax` is strictly positive but does not enforce an upper bound requiring the value to be less than 1.

In the `PayPublisherRespect` execution flow, the tax amount is calculated as `taxAmount = Tax * totalAmount`, and the publisher's payout is derived as `publisherAmount = totalAmount - taxAmount`. The transaction is rejected if `publisherAmount` is not strictly positive.

If governance configures the tax parameter to be greater than or equal to 1, any valid positive payment amount will result in a zero or negative `publisherAmount`.

Consequently, every `MsgPayPublisherRespect` transaction will deterministically fail with the error indicating that the publisher amount is not positive.

Recommendation

We recommend enforcing strict upper and lower bounds on the `tax` parameter within `validatePublisherRespectParams`, ensuring that the value satisfies $0 < \text{Tax} < 1$.

Status: Resolved

5. Event emission uses stale article ID resulting in incorrect observability data

Severity: Minor

In the `AddArticle` message handler within `x/cointrunk/keeper/msg_server.go`, a new `types.Article` is constructed with `Id` explicitly set to 0 and then passed to `SetArticle`. Inside `SetArticle`, the keeper detects that and assigns a new auto-incremented identifier before persisting the article in the KV store.

However, because the article struct is passed by value rather than by reference, the updated ID assigned within `SetArticle` is not reflected in the caller's copy.

Immediately after storage, the handler emits an `ArticleAddedEvent` using the original article variable, which still contains `Id` equal to 0.

As a result, every emitted event reports `ArticleId` equal to 0, despite the fact that the persisted article in state has a distinct, non-zero identifier.

This constitutes a logic and observability flaw. While the state remains correct, off-chain indexers, explorers, and client applications relying on emitted events to track newly created articles will be misled.

Recommendation

We recommend ensuring that the correct, auto-generated article ID is propagated back to the message handler before emitting the event.

Status: Resolved

6. Incomplete genesis validation allowing structurally invalid or inconsistent cointrunk module state

Severity: Minor

The `GenesisState.Validate` implementation in `x/cointrunk/types/genesis.go` only delegates validation to `gs.Params.Validate` and performs no checks on the remaining genesis state.

However, the module's `GenesisState` contains additional critical components, such as publishers, accepted domains, articles, and the articles counter, none of which are validated for structural correctness or internal consistency during chain initialization.

As a result, a genesis file may successfully pass validation while containing malformed or logically inconsistent data. For example, publishers may include invalid Bech32 addresses that would otherwise be rejected by `MsgSavePublisher.ValidateBasic`, accepted domains may not conform to the expected regex enforced by `MsgAcceptDomain`, and articles may contain URLs or picture URLs that would fail `MsgAddArticle.ValidateBasic` or domain validation checks. Additionally, duplicate keys or an `ArticlesCounter` value that does not match the highest stored article ID can be introduced without detection.

Starting the chain from such an invalid genesis state can lead to runtime panics (e.g., during address parsing), inconsistent query behavior, or invariant violations.

Recommendation

We recommend extending `GenesisState.Validate` to perform a comprehensive validation of all genesis state components.

Status: Acknowledged

7. Proposer influencable deterministic randomness enabling MEV and raffle outcome manipulation

Severity: Minor

The raffle randomness mechanism implemented in `x/burner/keeper/random_chance.go` derives a deterministic value by hashing a seed composed of `HeaderHash || AppHash || address`, applying SHA-256, and reducing the result modulo a fixed range before comparing it against `raffle.Chances`.

While the hash-to-range mapping itself is cryptographically sound, the entropy source is fully deterministic and primarily composed of block-local data that can be influenced by the current block proposer.

Both `HeaderHash` and, in particular, `AppHash` depend on the exact set and ordering of transactions included in the block. A rational validator can simulate multiple candidate block constructions locally, varying transaction ordering, inclusion, or even withholding and re-proposing blocks, to determine which configuration yields favorable raffle outcomes for their own addresses or colluding participants.

This undermines the probabilistic fairness guarantees of the raffle mechanism and exposes participants to economically motivated manipulation by validators.

Recommendation

We recommend replacing or augmenting the current entropy source with an unbiased randomness mechanism, for example, using a verifiable random function (VRF).

Status: Acknowledged

8. Misleading expiration check in raffle join logic

Severity: Minor

In `MsgJoinRaffle`, the inline comment states “*do not allow participants to join close to expiration*”, suggesting that the implementation prevents late-stage participation. However, the actual logic only rejects join attempts after the raffle has already expired.

The code retrieves the current raffle epoch using `GetRaffleCurrentEpoch` and rejects the transaction only if `re > 0 && raffle.EndAt <= (re-1)`.

In practice, this condition blocks joins only when the current epoch is strictly greater than the raffle’s `EndAt` value. As a result, participants are permitted to join throughout the entire final epoch, up until the epoch boundary where the cleanup hook executes and deletes the raffle.

Additionally, this validation also allows for the case where a user can join with many tickets into the future, where the raffle may expire.

This creates a documentation and intent mismatch. A reader may reasonably interpret the comment as indicating that participation is restricted near expiration (e.g., within the final epoch or some safety margin), when the implemented rule is effectively “no joins after expiration.”

Recommendation

We recommend either updating the comment to accurately reflect the current behavior (i.e., that joins are only disallowed after expiration) or tightening the condition to enforce a true “no joins close to expiration” policy if that was the intended requirement.

`JoinRaffle` should account for the number of tickets purchased, ensuring all tickets will be drawn before the raffle's cleanup epoch fires.

Status: Resolved

9. Incomplete genesis validation allowing malformed raffle and burn state initialization

Severity: Minor

In `x/burner/module/module.go`, the `ValidateGenesis` function unmarshals the module's genesis state and delegates validation entirely to `genState.Validate`.

However, the corresponding `GenesisState.Validate` implementation only validates module parameters and does not perform any consistency or structural checks on the remaining state, including raffles, participants, winners, and burned coin records.

As a result, the chain can successfully start from a genesis file containing malformed or internally inconsistent data. For example, raffles may contain empty or invalid denoms, zero or negative pot and ticket price values encoded as strings, or `EndAt` values already in the past. Participants or winners may reference non-existent raffles, and `BurnedCoins` entries may contain invalid or improperly formatted coin strings. Since these collections are not validated at initialization time, such inconsistencies are not detected during chain startup.

Multiple parts of the runtime logic assume that these invariants hold. The keeper logic parses pot and ticket price fields as positive integers, assumes referential integrity between raffles and winners or participants, and performs iteration and deletion operations during hooks and block lifecycle processing.

If malformed genesis data violates these assumptions, it may later trigger runtime panics, unexpected state transitions, or inconsistent query results.

Recommendation

We recommend extending `GenesisState.Validate` to comprehensively verify all state collections at initialization.

Status: Acknowledged

10. Unbounded `EndBlock` participant processing creating potential liveness risk

Severity: Minor

The `burner` module's `EndBlock` implementation invokes `WithdrawLuckyRaffleParticipants` on every block, wrapped in `ApplyFuncIfNoError`. This function loads all raffle participants scheduled for execution at the current block height and iterates over the entire set, performing multiple KV store reads and writes, SHA-256 and `big.Int` computations for randomness, bank transfers, and event emissions for each participant. There is no explicit upper bound on how many participants may share the same `ExecuteAt` height.

Because each `MsgJoinRaffle` can enqueue multiple participants and there is no per-height cap, it is theoretically possible for a large number of participants to be scheduled for the same future block. When that height is reached, `EndBlock` must process the entire set in a single execution. Since `EndBlock` runs in ABCI and outside normal per-transaction gas limits, a sufficiently large participant set could significantly increase block processing time.

In practice, the economic cost of tickets and the fact that participant entries may be staggered across blocks reduce the likelihood of large-scale exploitation under current assumptions.

However, the absence of explicit execution bounds means that future parameter changes, lower ticket costs, or altered queuing behavior could amplify the impact.

Recommendation

We recommend considering defensive design measures to ensure bounded execution per block. This may include implementing a per-height processing cap with batching across multiple blocks.

Status: Resolved

11. Epochs `BeginBlocker` processes only one tick per block, leaving epoch-dependent state stale after chain halt

Severity: Minor

In the `BeginBlocker` function in `x/epochs/keeper/abci.go:13-58` the `IterateEpochInfo` callback processes at most one epoch tick per identifier per block. The KV store iterator created at `x/epochs/keeper/store_epoch.go:69` visits each epoch key once; after `setEpochInfo` writes the updated epoch in `abci.go:53`, the iterator advances to the next identifier.

After a chain halt lasting N epoch durations, it takes N blocks to fully catch up. During this catch-up window, the `CurrentEpoch` value in the store is behind real time.

For example, the burner module's `JoinRaffle` handler uses `currentEpoch` to check raffle expiry, so users can join raffles that should already have expired. When the delayed cleanup hook fires for those epoch numbers during catch-up, it burns the remaining pot, including ticket funds paid by users who joined during the stale window. This compounds the ticket-loss finding, expanding the vulnerable window from just the ticket block spread to the entire catch-up period.

Examples where an incorrect epoch could be used:

- **Burner** `StartRaffle`: In `raffleFromMsgStartRaffle` in `x/burner/keeper/msg_raffle.go:74-75` the raffle `EndAt` is computed from the stale epoch count. A raffle started during catch-up has its effective duration

shortened by the number of missed epochs, and may expire immediately once the cleanup hook reaches the `EndAt` epoch during the same catch-up window.

- Rewards `ExitStaking` unlock: In `beginUnlock` in `x/rewards/keeper/msg_server_staking_reward.go:484-485` the `lockedUntil` target is computed from the stale epoch count. Tokens from an `ExitStaking` call during catch-up will unlock up to N hours early when the pending-unlock hook fires at that epoch number during catch-up.
- (Not in Scope) Rewards trading reward expiry: In `getNewTradingRewardExpireAt` in `x/rewards/keeper/keeper.go:104-108` the `ExpireAt` for trading rewards is computed from the stale epoch count, called by both `CreateTradingReward` in `x/rewards/keeper/msg_server_trading_reward.go:89` and `ActivateTradingReward` in `x/rewards/keeper/msg_server_gov.go:40`. A trading reward created or activated during catch-up may be prematurely expired and burned, or prematurely distributed, when the corresponding hook processes that epoch number during the same catch-up window.

Recommendation

We recommend changing the single-pass epoch processing to a loop that catches up all missed epochs within a single block, consistent with the standard Osmosis epochs implementation. A safety cap (200 iterations) can be added to prevent excessive processing after extremely long halts. Another approach is to identify operations that use the current epoch directly and block those when the chain is catching up.

Status: Resolved

12. Optional `TradeKeeper` dependency used without `nil` safety checks

Severity: Informational

In `x/burner/module/module.go`, the dependency injection wiring marks `TradeKeeper` as `optional:"true"` within `ModuleInputs`. However, the burner keeper and associated burn logic unconditionally invoke methods on `TradeKeeper`, including `IsNativeDenom`, `CanSwapForNativeDenom`, and `ModuleAddLiquidityWithNativeDenom`, without performing any `nil` checks.

If the module is initialized without a configured `TradeKeeper`, these method calls will result in a `nil` pointer dereference at runtime. This creates a latent misconfiguration risk where the application may compile and start successfully but panic during execution when burn logic is triggered.

Recommendation

We recommend either removing the `optional:"true"` designation to enforce mandatory dependency injection of `TradeKeeper`, or introducing explicit `nil` checks and fallback logic wherever `TradeKeeper` methods are invoked.

Status: Resolved

13. Incorrect telemetry metric key used in `EndBlock` instrumentation

Severity: Informational

In `x/burner/module/module.go`, the `EndBlock` implementation records execution timing using `telemetry.ModuleMeasureSince(types.ModuleName, time.Now(), telemetry.MetricKeyBeginBlocker)`.

However, this instrumentation is executed within the `EndBlock` method, while the metric key used corresponds to the begin-blocker.

This results in inaccurate telemetry labeling, as end-block execution time is incorrectly reported under the begin-blocker metric key.

Recommendation

We recommend replacing `telemetry.MetricKeyBeginBlocker` with the appropriate end-block metric key (e.g., `telemetry.MetricKeyEndBlocker`, if available) to ensure accurate instrumentation.

Status: Resolved

14. No minimum pot requirement allows cheap raffle squatting per denom

Severity: Informational

In the `StartRaffle` handler in `x/burner/keeper/msg_raftle.go:19-59` and the validation in `x/burner/types/message_start_raftle.go:48-55`, the pot amount only requires `IsPositive`, allowing a pot as small as 1 unit.

Combined with the one-affle-per-denom constraint in lines 26-29 of `msg_raftle.go` and a maximum duration of 180 days, an attacker can cheaply occupy any denom's raffle slot for an extended period, blocking legitimate raffles from being created.

In practice, the impact is limited since the raffle still functions normally, losers' ticket prices accumulate into the pot, so the raffle becomes viable after a few participants, regardless of the initial pot size, but the raffle will start at a very low pot and attract fewer users.

Recommendation

We recommend adding a minimum pot parameter, governance-configurable, or charging a creation fee to make raffle squatting economically impractical.

Status: Resolved

15. Raffle participant counter tracks tickets instead of unique participants

Severity: Informational

In the `SetRaffleParticipant` function in `x/burner/keeper/store_raffle.go:147-155` the `RaffleParticipantCounter` is incremented once per ticket rather than once per unique participant.

The counter is only used as a monotonic nonce to generate unique KV store keys and is not exposed as a participant count metric, so there is no functional impact.

However, the naming is misleading and could cause confusion during future development.

Recommendation

We recommend renaming `RaffleParticipantCounter` to `RaffleTicketCounter` to better reflect its purpose.

Status: Acknowledged

16.No article URL uniqueness check allows anonymous duplicate spam

Severity: Informational

In the `AddArticle` handler in `x/cointrunk/keeper/msg_server.go:27-72` and `SetArticle` in `x/cointrunk/keeper/store_article.go:26-42`, there is no check for URL uniqueness before storing an article. The same URL can be posted an unlimited number of times, each receiving a distinct auto-incremented ID.

While anonymous users must pay `AnonArticleCost` per article, a motivated griefer could spam duplicate articles to clutter the cointrunk module interface and degrade user experience. This could also exhaust the global monthly `AnonArticleLimit`, denying other anonymous users the ability to post for the remainder of the period.

Recommendation

We recommend enforcing URL uniqueness for anonymous articles to prevent duplicate submissions and quota waste.

Status: Acknowledged